

STRUCTURED QUERY LANGUAGE(SQL)

CREATE USER WITH PASSWORD IN SQL

CONNECT SYSTEM/MANAGER

CREATE USER PRIYANKA IDENTIFIED BY PRIYANKA123;

O/P : USER CREATED

GRANT CONNECT,RESOURCE,DBA TO PRIYANKA;

GRANT SUCCEEDED.

EXIT

CONNECT TO SYSTEM WITH USER

CONNECT SYSTEM/MANAGER

CONNECTED

CONNECT PRIYANKA/PRIYANKA123

CONNECTED

TABLE 1: CREATE TABLE STUDENT

```
SQL>CREATE TABLE STUDENT(  
ROLLNO INT,  
MARKS INT,  
NAME VARCHAR(10),  
CITY VARCHAR(10)  
);
```

O/P: TABLE CREATED

TO DESCRIBE THE TABLE FOR SHOW THE STRUCTURE OF STUDENT TABLE

```
DESC STUDENT;
```

```
(DESCRIBE STUDENT)
```

Name	Null? Type
-----	-----
ROLLNO	NUMBER(38)
MARKS	NUMBER(38)
NAME	VARCHAR2(10)
CITY	VARCHAR2(10)

ALTER COMMAND

```
ALTER TABLE STUDENT ADD (ADDRESS VARCHAR2(30));
```

O/P : Table altered.

DESC STUDENT;

Name	Null? Type
-----	-----
ROLLNO	NUMBER(38)
MARKS	NUMBER(38)
NAME	VARCHAR2(10)
CITY	VARCHAR2(10)
ADDRESS	VARCHAR2(30)

ALTER TABLE STUDENT MODIFY (ADDRESS VARCHAR2(40));

O/P: TABLE ALTERED.

DESC STUDENT;

Name	Null? Type
-----	-----
ROLLNO	NUMBER(38)
MARKS	NUMBER(38)
NAME	VARCHAR2(10)
CITY	VARCHAR2(10)
ADDRESS	VARCHAR2(40)

ALTER TABLE STUDENT DROP (ADDRESS);

O/P:

DESC STUDENT;

Name	Null? Type
------	------------

ROLLNO	NUMBER(38)
MARKS	NUMBER(38)
NAME	VARCHAR2(10)
CITY	VARCHAR2(10)

DROP TABLE STUDENT;

Table dropped.

CREATE TABLE STUDENT

```
SQL>CREATE TABLE STUDENT(  
ROLLNO INT,  
MARKS INT,  
NAME VARCHAR(10),  
CITY VARCHAR(10)  
);
```

INSERT COMMAND(METHOD 1)

```
INSERT INTO STUDENT VALUES(101,55,'RAMESH', 'BENGALORE');  
INSERT INTO STUDENT VALUES(102,75,'RAM','BALLARI');  
INSERT INTO STUDENT VALUES(103,86,'RAVI','BELAGAVI');
```

INSERT COMMAND (METHOD 2)

```
INSERT INTO STUDENT (ROLLNO,MARKS,NAME,CITY) VALUES(104,88,'DHANYA','HUBBALI');  
INSERT INTO STUDENT (ROLLNO,MARKS,NAME,CITY) VALUES(105,98,'DHARUVA','MANDYA');  
INSERT INTO STUDENT (ROLLNO,MARKS,NAME,CITY) VALUES(106,78,'MANU','BALLARI');
```

VIEW THE OUT PUT:

SELECT *FROM STUDENT;

ROLLNO	MARKS	NAME	CITY
101	55	RAMESH	BENGALORE
102	75	RAM	BALLARI
103	86	RAVI	BELAGAVI
104	88	DHANYA	HUBBALI
105	98	DHRUVA	MANDYA
106	78	MANU	BALLARI

6 rows selected

UPDATE COMMAND

UPDATE STUDENT SET MARKS=50 WHERE NAME='RAVI';

1 row updated.

UPDATE STUDENT SET ROLLNO=201 WHERE NAME='RAVI';

1 row updated.

VIEW THE OUTPUT:

SELECT *FROM STUDENT;

ROLLNO	MARKS	NAME	CITY
101	55	RAMESH	BENGALORE
102	75	RAM	BALLARI
201	50	RAVI	BELAGAVI

104 88 DHANYA HUBBALI

105 98 DHRUVA MANDYA

106 78 MANU BALLARI

6 rows selected.

UPDATE COMMAND

UPDATE STUDENT SET MARKS=50 WHERE NAME='RAVI';

SELECT * FROM STUDENT;

ROLLNO	MARKS	NAME	CITY
101	55	RAMESH	BENGALORE
102	75	RAM	BALLARI
201	50	RAVI	BELAGAVI
104	88	DHANYA	HUBBALI
105	98	DHRUVA	MANDYA
106	78	MANU	BALLARI

UPDATE STUDENT SET NAME='MBA' WHERE ROLLNO=101;

1 row updated.

O/P:

SQL> SELECT * FROM STUDENT;

ROLLNO	MARKS	NAME	CITY
101	55	MBA	BENGALORE

102 75 RAM BALLARI

201 50 RAVI BELAGAVI

104 88 DHANYA HUBBALI

105 98 DHRUVA MANDYA

106 78 MANU BALLARI

6 rows selected.

DELETE COMMAND

DELETE STUDENT WHERE ROLLNO=201;

1 row deleted.

SQL> SELECT * FROM STUDENT;

ROLLNO	MARKS	NAME	CITY
101	55	MBA	BENGALORE
102	75	RAM	BALLARI
104	88	DHANYA	HUBBALI
105	98	DHRUVA	MANDYA
106	78	MANU	BALLARI

SQL> DELETE STUDENT WHERE NAME='MANU';

1 row deleted.

SELECT COMMAND

SELECT COLUMN1,COLUMN2,COLUMNN FROM TABLE_NAME;

[WHERE CONDITION(S)]

[GROUPBY COLUMN-LIST]

[HAVING CONDITION(S)]

[ORDER BY COLUMN-NAME(S)];

SELECT COMMAND

SELECT*FROM STUDENT;

SELECT*FROM STUDENT WHERE MARKS>50;

SELECT NAME FROM STUDENT WHERE MARKS>50;

SELECT NAME, ROLLNO FROM STUDENT WHERE MARKS>50;

SELECT*FROM STUDENT WHERE MARK

ROLLNO	MARKS	NAME	CITY
--------	-------	------	------

-----	-----	-----	-----
-------	-------	-------	-------

102	75	RAM	BALLARI
-----	----	-----	---------

104	88	DHANYA	HUBBALI
-----	----	--------	---------

105	98	DHRUVA	MANDYA
-----	----	--------	--------

106	78	MANU	BALLARI
-----	----	------	---------

SQL> SELECT NAME FROM STUDENT WHERE MARKS>50;

NAME

RAM

DHANYA

DHRUVA

MANU

SELECT NAME FROM STUDENT WHERE CITY='BALLARI';

O/P:

NAME

RAM

MANU

SELECT*FROM STUDENT ORDER BY NAME;

ROLLNO	MARKS	NAME	CITY
--------	-------	------	------

104	88	DHANYA	HUBBALI
-----	----	--------	---------

105	98	DHRUVA	MANDYA
-----	----	--------	--------

106	78	MANU	BALLARI
-----	----	------	---------

102	75	RAM	BALLARI
-----	----	-----	---------

SELECT*FROM STUDENT ORDER BY NAME DESC;

ROLLNO	MARKS	NAME	CITY
--------	-------	------	------

102	75	RAM	BALLARI
-----	----	-----	---------

106 78 MANU BALLARI

105 98 DHRUVA MANDYA

104 88 DHANYA HUBBALI

SELECT*FROM STUDENT WHERE MARKS>50 ORDER BY MARKS DESC;

ROLLNO	MARKS	NAME	CITY
--------	-------	------	------

105	98	DHRUVA	MANDYA
-----	----	--------	--------

104	88	DHANYA	HUBBALI
-----	----	--------	---------

106	78	MANU	BALLARI
-----	----	------	---------

102	75	RAM	BALLARI
-----	----	-----	---------

COMMIT;

Commit complete.

TABLE 2: CREATE TABLE STUD

```
CREATE TABLE STUD(  
ROLLNO INT,  
NAME VARCHAR(10)  
);  
DESC STUD;
```

INSERT VALUES

```
INSERT INTO STUD VALUES(1,'KIRAN');  
INSERT INTO STUD VALUES(2,'PAVAN');
```

TASK1: PRIMARY KEY :

PRIMARY KEY: NOT NULL+UNIQUE

```
CREATE TABLE STUD(  
RNO NUMBER PRIMARY KEY,  
NAME VARCHAR(10));
```

TABLE CREATED

INSERT VALUES:

```
INSERT INTO STUD VALUES(1,'KIRAN');  
INSERT INTO STUD VALUES(2,'PAVAN');  
INSERT INTO STUD VALUES(1,'KIRAN');  
  
1 row created.
```

```
SQL> INSERT INTO STUD VALUES(2,'PAVAN');
```

1 row created.

O/P:

```
SELECT*FROM STUD;
```

```
RNO NAME
```

```
-----
```

```
1 KIRAN
```

```
2 PAVAN
```

TASK2 :FOREIGN KEY

FOREIGN KEY CONSTARINT:

```
CREATE TABLE DEPT
```

```
(
```

```
DEPTID INT PRIMARY KEY,
```

```
DNAME VARCHAR(10));
```

INSERT VALUES

```
INSERT INTO DEPT
```

```
VALUES(100,'CSE');
```

```
INSERT INTO DEPT
```

```
VALUES(200,'MBA');
```

```
INSERT INTO DEPT
```

```
VALUES(300,'MECH');
```

O/P:

```
SELECT*FROM DEPT;
```

```
DEPTID DNAME
```

```
-----
```

```
100 CSE
```

```
200 MBA
```

```
300 MECH
```

CREATE TABLE STUD

```
(
```

```
RNO INT PRIMARY KEY,
```

```
NAME VARCHAR(10),
```

```
DID INT,
```

```
FOREIGN KEY (DID) REFERENCES
```

```
DEPT(DEPTID) ON DELETE CASCADE);
```

INSERT INTO STUD VALUES:

```
INSERT INTO STUD VALUES(1,'KIRAN',100);
```

```
INSERT INTO STUD VALUES(2,'PAVAN',200);
```

```
INSERT INTO STUD VALUES(3,'XYZ',200);
```

```
INSERT INTO STUD VALUES(4,'ABC',300);
```

O/P:

```
SELECT*FROM STUD;
```

```
RNO NAME
```

```
DID
```

1 KIRAN	100
2 PAVAN	200
3 XYZ	200
4 ABC	300

SQL> INSERT INTO STUD VALUES(10,'RAM',900);

ERROR at line 1:

ORA-02291: integrity constraint (REKHA.SYS_C003997) violated - parent key not found

TASK 1:

TABLE 3: CREATE TABLE SALESMAN

```
CREATE TABLE SALESMAN(  
2 SALESMAN_ID INTEGER PRIMARY KEY,  
3 NAME VARCHAR(20),  
4 CITY VARCHAR(20),  
5 COMMISSION VARCHAR(20));
```

Table created.

CUSTOMER:

```
CREATE TABLE CUSTOMER(  
CUSTOMER_ID INTEGER PRIMARY KEY,  
CUST_NAME VARCHAR(20),  
CITY VARCHAR(20),  
GRADE INTEGER,  
SALESMAN_ID INTEGER,  
FOREIGN KEY(SALESMAN_ID)REFERENCES  
SALESMAN(SALESMAN_ID) ON DELETE SET NULL);
```

ORDERS:

```
CREATE TABLE ORDERS(  
ORDER_ID INTEGER PRIMARY KEY,  
PURCHASE_AMOUNT DECIMAL(10,2),
```

```

ORDER_DATE DATE,
CUSTOMER_ID INTEGER,
SALESMAN_ID INTEGER,
FOREIGN KEY(CUSTOMER_ID) REFERENCES
CUSTOMER(CUSTOMER_ID) ON DELETE CASCADE,
FOREIGN KEY (SALESMAN_ID) REFERENCES
SALESMAN(SALESMAN_ID) ON DELETE CASCADE);

```

DESC SALESMAN;

Name	Null?	Type
SALESMAN_ID	NOT NULL	NUMBER(38)
NAME		VARCHAR2(20)
CITY		VARCHAR2(20)
COMISSION		VARCHAR2(20)

```
SQL> DESC CUSTOMER;
```

Name	Null?	Type
CUSTOMER_ID	NOT NULL	NUMBER(38)
CUST_NAME		VARCHAR2(20)
CITY		VARCHAR2(20)
GRADE		NUMBER(38)
SALESMAN_ID		NUMBER(38)

SQL> DESC ORDERS;

Name	Null?	Type
ORDER_ID	NOT NULL	NUMBER(38)
PURCHASE_AMOUNT		NUMBER(10,2)
ORDER_DATE		DATE
CUSTOMER_ID		NUMBER(38)
SALESMAN_ID		NUMBER(38)

INSERT INTO SALESMAN VALUES :

INSERT INTO SALESMAN VALUES(1000,'RAHUL','BANGALORE','20%');
INSERT INTO SALESMAN VALUES(2000,'ANKITA','BANGALORE','25%');
INSERT INTO SALESMAN VALUES(3000,'SHARMA','MYSORE','30%');
INSERT INTO SALESMAN VALUES(4000,'ANJALI','DELHI','15%'); INSERT
INTO SALESMAN VALUES(5000,'RAJ','HYDERBAD','15%');

SELECT*FROM SALESMAN;

SALESMAN_ID	NAME	CITY	COMISSION
1000	RAHUL	BANGALORE	20%
2000	ANKITA	BANGALORE	25%
3000	SHARMA	MYSORE	30%
4000	ANJALI	DELHI	15%
5000	RAJ	HYDERBAD	15%

INSERT INTO CUSTOMER VALUES :

```
INSERT INTO CUSTOMER VALUES(1,'ADAY','BANGALORE',100,1000);
INSERT INTO CUSTOMER VALUES(2,'BANU','MANGALORE',300,1000);
INSERT INTO CUSTOMER VALUES(3,'CHETAN','CHENNAI',400,2000);
INSERT INTO CUSTOMER VALUES(4,'DANISH','BANGALORE',200,2000);
INSERT INTO CUSTOMER VALUES(5,'ESHA','BANGALORE',400,3000);
```

```
SELECT*FROM CUSTOMER;
```

CUSTOMER_ID	CUST_NAME	CITY	GRADE	SALESMAN_ID
1	ADAY	BANGALORE	100	1000
2	BANU	MANGALORE	300	1000
3	CHETAN	CHENNAI	400	2000
4	DANISH	BANGALORE	200	2000
5	ESHA	BANGALORE	400	3000

INSERT INTO ORDERS VALUES :

```
INSERT INTO ORDERS VALUES(201,5000,'02-JAN-2020',1,1000);
INSERT INTO ORDERS VALUES(202,450,'09-APR-2020',1,2000);
INSERT INTO ORDERS VALUES(203,1000,'15-MAR-2020',3,2000);
INSERT INTO ORDERS VALUES(204,3500,'09-JUL-2020',4,3000);
```

INSERT INTO ORDERS VALUES(205,550,'05-MAY-2020',2,2000);

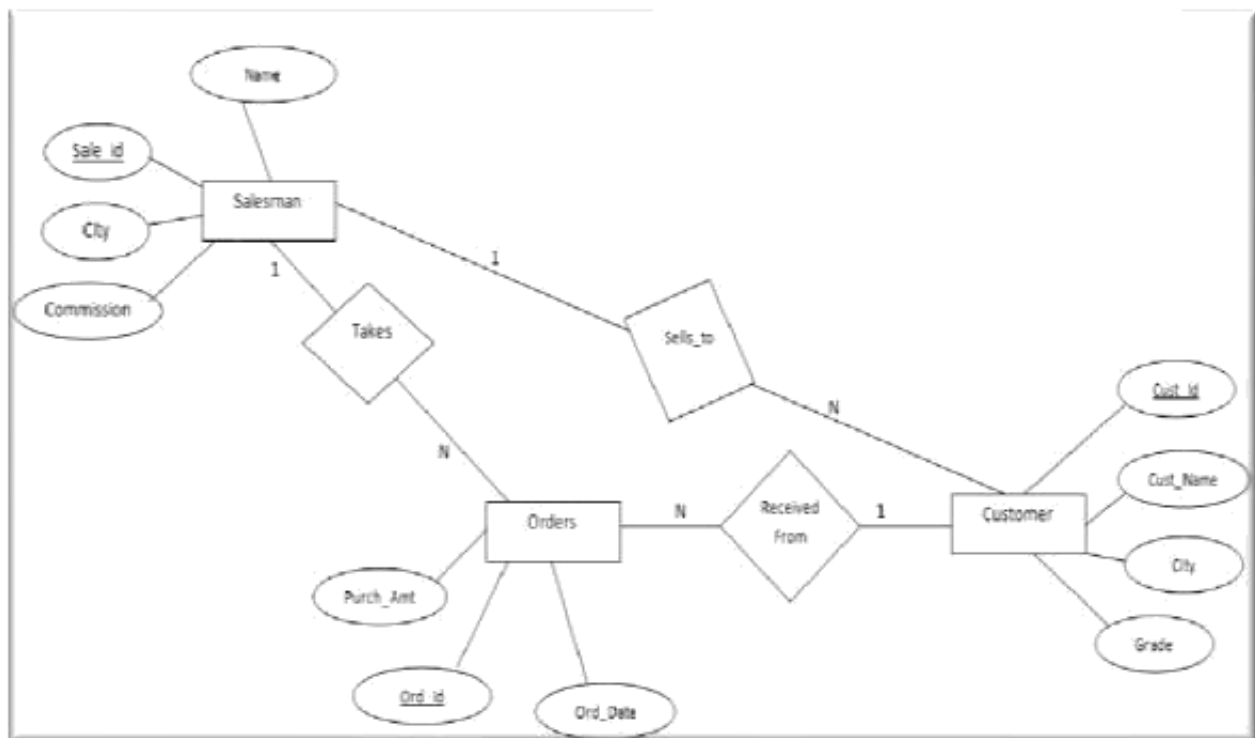
INSERT INTO ORDERS VALUES(206,6000,'02-JUN-2020',3,1000);

SELECT*FROM ORDERS;

ORDER_ID PURCHASE_AMOUNT ORDER_DAT CUSTOMER_ID SALESMAN_ID

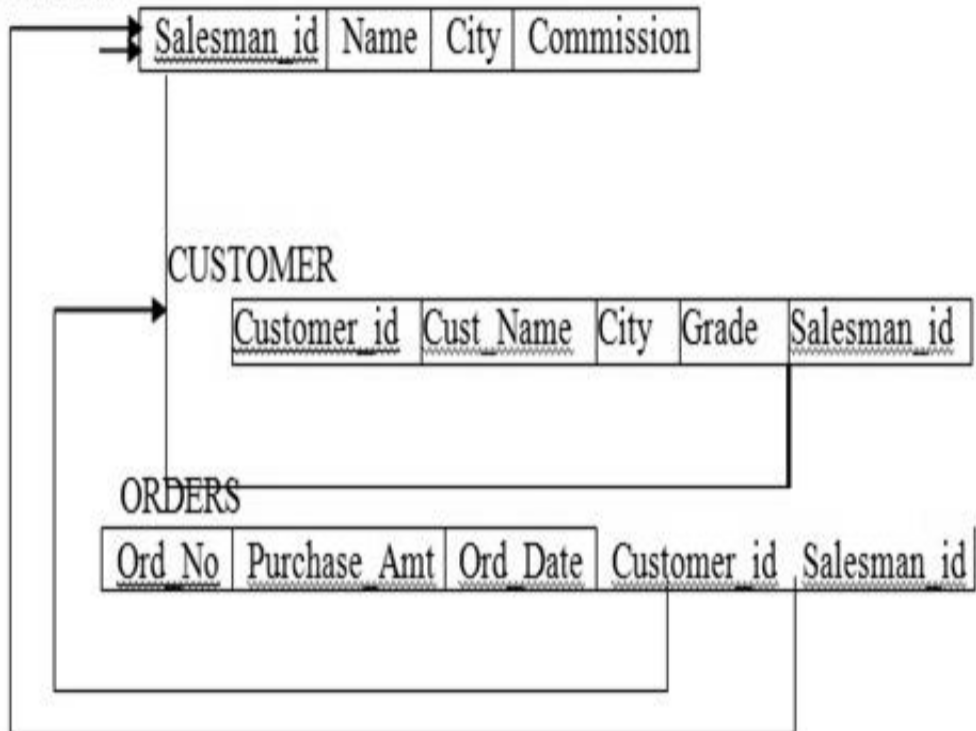
ORDER_ID	PURCHASE_AMOUNT	ORDER_DAT	CUSTOMER_ID	SALESMAN_ID
201	5000	02-JAN-20	1	1000
202	450	09-APR-20	1	2000
203	1000	15-MAR-20	3	2000
204	3500	09-JUL-20	4	3000
205	550	05-MAY-20	2	2000
206	6000	02-JUN-20	3	1000

ER DIAGRAM :



Step 1: SCHEMA DIAGRAM

SALESMAN



DAY—2:

```
SELECT*FROM STUDENT;
ROLLNO  MARKS NAME  CITY
-----
102    75 RAM    BALLARI
104    88 DHANYA  HUBBALI
105    98 DHRUVA  MANDYA
106    78 MANU    BALLARI
```

COUNT(ROLLNO)FROM STUDENT:

- SQL> SELECT COUNT(ROLLNO)FROM STUDENT;

COUNT(ROLLNO)

4

- SQL> SELECT COUNT(ROLLNO) FROM STUDENT WHERE CITY='BALLARI';

COUNT(ROLLNO)

2

AVG(MARKS) FROM STUDENT:

SQL> SELECT AVG(MARKS) FROM STUDENT;

AVG(MARKS)

84.75

SELECT SUM(MARKS) FROM STUDENT:

SQL> SELECT SUM(MARKS) FROM STUDENT;

SUM(MARKS)

339

MAX:

SQL> SELECT MAX (MARKS) FROM STUDENT;

MAX(MARKS)

98

MIN:

SQL> SELECT MIN(MARKS) FROM STUDENT;

MIN(MARKS)

75

GROUP BY COUNT:

SQL> SELECT COUNT(ROLLNO),CITY FROM STUDENT GROUP BY CITY;

COUNT(ROLLNO) CITY

1 MANDYA

1 HUBBALI

2 BALLARI

GROUP BY AVG :

SQL> SELECT CITY,AVG(MARKS) FROM STUDENT GROUP BY CITY;

CITY AVG(MARKS)

MANDYA 98

HUBBALI 88

BALLARI 76.5

GROUP BY OF MAX:

SQL> SELECT CITY, MAX(MARKS) FROM STUDENT GROUP BY CITY;

CITY MAX(MARKS)

MANDYA 98

HUBBALI 88

BALLARI 78

GROUP BY AVG:

SQL> SELECT CITY, AVG(MARKS) FROM STUDENT GROUP BY CITY;

CITY AVG(MARKS)

MANDYA 98

88 HUBBALI

76.5 BALLARI

1. DISPLAY THE NAMES WHO SCORES MAX MARKS

SQL> SELECT NAME FROM STUDENT WHERE MARKS=(SELECT MAX(MARKS) FROM STUDENT);

NAME

DHRUVA

2. DISPLAY THE NAMES WHO SCORES MIN MARKS

SQL> SELECT NAME FROM STUDENT WHERE MARKS=(SELECT MIN(MARKS) FROM STUDENT);

NAME

RAM

3. **DISPLAY THE NAME WHO LIVED IN BALLARI**

SQL> SELECT NAME FROM STUDENT WHERE CITY='BALLARI';

NAME

RAM

MANU

6, CLAUSES:

- SELECT-
- FROM- WHICH TABLE
- WHERE- CONDITION
- GROUP BY – MAKING GROUP THE EMPLOYEES BY DEPARTMENT
- ORDER BY -
- HAVING – HAVING ALSO PUTTING THE CONDITION (HAVING TOTAL_SALARY>100000;)

TASK 2:

TABLE 4: CREATING EMPLOYEE TABLE

```
CREATE TABLE EMPLOYEES(  
    ID INT PRIMARY KEY,  
    NAME VARCHAR(20),  
    SALARY DECIMAL (10,2),  
    DEPARTMENT_ID INT  
);
```

```
CREATE TABLE DEPARTMENT(  
    ID INT PRIMARY KEY,  
    DEPARTMENT_NAME VARCHAR(20)  
);
```

```
INSERT INTO DEPARTMENT VALUES(1,'HR');
```

```
INSERT INTO DEPARTMENT VALUES(2,'IT');
```

```
INSERT INTO DEPARTMENT VALUES(3,'FINANCE');
```

```
INSERT INTO EMPLOYEES VALUES (1,'ALICE',70000,1);
```

```
INSERT INTO EMPLOYEES VALUES (2,'BOB',750000,2);
```

```
INSERT INTO EMPLOYEES VALUES (3,'CHARLIE',60000,2);
```

```
INSERT INTO EMPLOYEES VALUES (4,'DAVID',80000,3);
```

```
INSERT INTO EMPLOYEES VALUES (5,'EVA',90000,3);
```

```
INSERT INTO EMPLOYEES VALUES (6,'FRANK',55000,1);
```

SELECT*FROM EMPLOYEES;

ID	NAME	SALARY	DEPARTMENT_ID
1	ALICE	70000	1
2	BOB	750000	2
3	CHARLIE	60000	2
4	DAVID	80000	3
5	EVA	90000	3
6	FRANK	55000	1

6 rows selected.

2. WHERE

SQL> SELECT NAME,SALARY FROM EMPLOYEES WHERE SALARY>50000;

NAME	SALARY
ALICE	70000
BOB	750000
CHARLIE	60000
DAVID	80000
EVA	90000
FRANK	55000

6 rows selected.

SQL> SELECT NAME,SALARY FROM EMPLOYEES WHERE SALARY>50000;

NAME	SALARY
------	--------

ALICE	70000
BOB	750000
CHARLIE	60000
DAVID	80000
EVA	90000
FRANK	55000

6 rows selected.

GROUP BY:

SQL> SELECT DEPARTMENT_ID, SUM(SALARY) AS TOTAL_SALARY FROM EMPLOYEES GROUP BY DEPARTMENT_ID;

DEPARTMENT_ID TOTAL_SALARY

1	125000
2	810000
3	170000

SQL> SELECT NAME, SALARY FROM EMPLOYEES WHERE SALARY > 90000;

NAME SALARY

BOB	750000
-----	--------

4.HAVING:

SQL> SELECT DEPARTMENT_ID, SUM(SALARY) AS TOTAL_SALARY FROM EMPLOYEES GROUP BY DEPARTMENT_ID HAVING SUM(SALARY) > 150000;

DEPARTMENT_ID TOTAL_SALARY

2	810000
3	170000

5.ORDER BY

SELECT NAME,SALARY FROM EMPLOYEES ORDER BY SALARY DESC;

NAME	SALARY
BOB	750000
EVA	90000
DAVID	80000
ALICE	70000
CHARLIE	60000
FRANK	55000

6 rows selected.

SELECT NAME,SALARY FROM EMPLOYEES WHERE SALARY>50000;

1.COUNT THE CUSTOMER WITH GRADES ABOVE BANGALORE'S AVG

SELECT GRADE,COUNT(DISTINCT CUSTOMER_ID)
FROM CUSTOMER
GROUP BY GRADE
HAVING GRADE>(SELECT AVG(GRADE)
FROM CUSTOMER
WHERE CITY='BANGALORE');

GRADE COUNT(DISTINCTCUSTOMER_ID)

300	1
400	2

2.FIND THE NAME AND NUMBER OF ALL SALESMAN WHO HAD MORE THAN ONE CUSTOMER

SELECT SALESMAN_ID,NAME
FROM SALESMAN S
WHERE (SELECT COUNT(*)
FROM CUSTOMER C

WHERE C.SALESMAN_ID=S.SALESMAN_ID)>1;

:SALESMAN_ID NAME

1000 RAHUL

2000 ANKITA

3.LIST ALL THE SALESMAN AND INDICATE THOSE WHO HAVE AND DON'T HAVE CUSTOMERS IN THEIR CITIES(USE UNION OPERATION.)

SELECT S.SALESMAN_ID,S.NAME,C.CUST_NAME,S.COMISSION
FROM SALESMAN S,CUSTOMER C WHERE S.CITY=C.CITY

UNION

SELECT S.SALESMAN_ID ,S.NAME,'NO MATCH',S.COMISSION FROM SALESMAN S
WHERE CITY NOT IN (SELECT CITY FROM CUSTOMER)
ORDER BY 1 ASC;

O/P:

SALESMAN_ID NAME CUST_NAME COMISSION

1000 RAHUL	ADAY	20%
1000 RAHUL	DANISH	20%
1000 RAHUL	ESHA	20%
2000 ANKITA	ADAY	25%
2000 ANKITA	DANISH	25%
2000 ANKITA	ESHA	25%
3000 SHARMA	NO MATCH	30%
4000 ANJALI	NO MATCH	15%
5000 RAJ	NO MATCH	15%

4.Create a view that finds the salesman who has the customer with the highest order of a day.

CREATE VIEW V_SALESMAN AS
SELECT O.ORDER_DATE,S.SALESMAN_ID,S.NAME
FROM SALESMAN S,ORDERS O
WHERE S.SALESMAN_ID = O.SALESMAN_ID AND O.PURCHASE_AMOUNT =(SELECT
MAX(PURCHASE_AMOUNT)
FROM ORDERS C

WHERE C.ORDER_DATE = O.ORDER_DATE);

O/P:

VIEW CREATED

SELECT * FROM V_SALESMAN;

SQL> SELECT * FROM V_SALESMAN;

ORDER_DAT SALESMAN_ID NAME

ORDER_DAT	SALESMAN_ID	NAME
02-JAN-20	1000	RAHUL
09-APR-20	2000	ANKITA
15-MAR-20	2000	ANKITA
09-JUL-20	3000	SHARMA
05-MAY-20	2000	ANKITA
02-JUN-20	1000	RAHUL

5.Demonstrate the DELETE operation by removing salesman with id 1000. All his orders must also be deleted

DELETE FROM SALESMAN WHERE SALESMAN_ID=1000;

O/P:

SELECT*FROM SALESMAN;

SALESMAN_ID	NAME	CITY	COMISSION
2000	ANKITA	BANGALORE	25%
3000	SHARMA	MYSORE	30%
4000	ANJALI	DELHI	15%
5000	RAJ	HYDERBAD	15%

SQL> SELECT *FROM CUSTOMER;

O/P:

CUSTOMER_ID	CUST_NAME	CITY	GRADE	SALESMAN_ID
1	ADAY	BANGALORE	100	

2 BANU	MANGALORE	300	
3 CHETAN	CHENNAI	400	2000
4 DANISH	BANGALORE	200	2000
5 ESHA	BANGALORE	400	3000

TABLE 5:

MOVIE DATABASE

1. **ACTOR**

2. **DIRECTOR**

3. **MOVIES**

4. **MOVIE_CAST**

5. **RATING**

STEP1: DESIGN A SCHEMA DIGRAM

STEP2: CREATING TABLES

ACTOR:

```
CREATE TABLE ACTOR(  
  ACT_ID NUMBER (3),  
  ACT_NAME VARCHAR (20),  
  ACT_GENDER CHAR (1),  
  PRIMARY KEY (ACT_ID));
```

TABLE CREATED

```
SELECT* FROM ACTOR
```

ACT_ID	ACT_NAME	A
301	ANUSHKA	F
302	PRABHAS	M
303	PUNITH	M
304	JERMY	M

DIRECTOR:

```
CREATE TABLE DIRECTOR(  
  DIR_ID NUMBER (3),  
  DIR_NAME VARCHAR (20),  
  DIR_PHONE NUMBER (10),  
  PRIMARY KEY (DIR_ID));
```


TABLE CREATED

SQL> SELECT*FROM DIRECTOR;

DIR_ID	DIR_NAME	DIR_PHONE
60	RAJAMOULI	8751611001
61	HITCHCOCK	7766138911
62	FARAN	9986776531
63	STEVEN SPIELBERG	8989776530

MOVIES:

```
CREATE TABLE MOVIES(  
MOV_ID NUMBER (4),  
MOV_TITLE VARCHAR (25),  
MOV_YEAR NUMBER (4),  
MOV_LANG VARCHAR (12),  
DIR_ID NUMBER (3), PRIMARY KEY (MOV_ID),  
FOREIGN KEY (DIR_ID) REFERENCES DIRECTOR (DIR_ID));
```

SQL> SELECT* FROM MOVIES;

MOV_ID	MOV_TITLE	MOV_YEAR	MOV_LANG	DIR_ID
1001	BAHUBALI-2	2017	TELAGU	60
1002	BAHUBALI-1	2015	TELAGU	60
1003	AKASH	2008	KANNADA	61
1004	WAR HORSE	2011	ENGLISH	63
1005	FF6	2011	ENGLISH	63

MOVIE_CAST:

```
CREATE TABLE MOVIE_CAST(  
ACT_ID NUMBER (3),  
MOV_ID NUMBER (4),  
ROLE VARCHAR (10),  
PRIMARY KEY (ACT_ID, MOV_ID),
```

```
FOREIGN KEY (ACT_ID) REFERENCES ACTOR (ACT_ID),
FOREIGN KEY (MOV_ID) REFERENCES MOVIES (MOV_ID));
SQL> SELECT*FROM MOVIE_CAST;
```

ACT_ID	MOV_ID	ROLE
301	1002	HEROINE
301	1001	HEROINE
303	1003	HERO
303	1002	GUEST
304	1004	HERO

RATING:

```
CREATE TABLE RATING(
MOV_ID NUMBER (4),
REV_STARS VARCHAR (25),
PRIMARY KEY (MOV_ID),
FOREIGN KEY (MOV_ID) REFERENCES MOVIES (MOV_ID));
```

```
SQL> SELECT*FROM RATING;
```

MOV_ID	REV_STARS
1001	4
1002	2
1003	5
1004	5
1005	5

STEP3: INSERTING VALUES TO TABLES:

INSERTING ACTOR VALUES:

```
INSERT INTO ACTOR VALUES (301,'ANUSHKA','F');
INSERT INTO ACTOR VALUES (302,'PRABHAS','M');
INSERT INTO ACTOR VALUES (303,'PUNITH','M');
INSERT INTO ACTOR VALUES (304,'JERMY','M');
```

O/P:

SQL> SELECT * FROM ACTOR;

ACT_ID	ACT_NAME	A
301	ANUSHKA	F
302	PRABHAS	M
303	PUNITH	M
304	JERMY	M

INSERT DIRECTOR VALUES:

```
INSERT INTO DIRECTOR VALUES (60,'RAJAMOULI', 8751611001);
INSERT INTO DIRECTOR VALUES (61,'HITCHCOCK', 7766138911);
INSERT INTO DIRECTOR VALUES (62,'FARAN', 9986776531);
INSERT INTO DIRECTOR VALUES (63,'STEVEN SPIELBERG', 8989776530);
```

O/P:

SQL> SELECT * FROM DIRECTOR;

DIR_ID	DIR_NAME	DIR_PHONE
60	RAJAMOULI	8751611001
61	HITCHCOCK	7766138911
62	FARAN	9986776531
63	STEVEN SPIELBERG	8989776530

INSERT MOVIES VALUES:

```
INSERT INTO MOVIES VALUES (1001,'BAHUBALI-2', 2017, 'TELUGU', 60);
INSERT INTO MOVIES VALUES (1002,'BAHUBALI-1', 2015, 'TELUGU', 60);
INSERT INTO MOVIES VALUES (1003,'AKASH', 2008, 'KANNADA', 61);
INSERT INTO MOVIES VALUES (1004,'WAR HORSE', 2011, 'ENGLISH', 63);
INSERT INTO MOVIES VALUES (1005,'FF6', 2011, 'ENGLISH', 63);
```

O/P:

SQL> SELECT * FROM MOVIES;

MOV_ID	MOV_TITLE	MOV_YEAR	MOV_LANG	DIR_ID
--------	-----------	----------	----------	--------

1001 BAHUBALI-2	2017 TELAGU	60
1002 BAHUBALI-1	2015 TELAGU	60
1003 AKASH	2008 KANNADA	61
1004 WAR HORSE	2011 ENGLISH	63
1005 FF6	2011 ENGLISH	63

INSERT MOVIE_CAST VALUES:

```
INSERT INTO MOVIE_CAST VALUES (301, 1002, 'HEROINE');
INSERT INTO MOVIE_CAST VALUES (301, 1001, 'HEROINE');
INSERT INTO MOVIE_CAST VALUES (303, 1003, 'HERO');
INSERT INTO MOVIE_CAST VALUES (303, 1002, 'GUEST');
INSERT INTO MOVIE_CAST VALUES (304, 1004, 'HERO');
```

O/P:

```
SQL> SELECT * FROM MOVIE_CAST;
```

ACT_ID	MOV_ID	ROLE
301	1002	HEROINE
301	1001	HEROINE
303	1003	HERO
303	1002	GUEST
304	1004	HERO

INSERT RATING VALUES:

```
INSERT INTO RATING VALUES (1001, 4);
INSERT INTO RATING VALUES (1002, 2);
INSERT INTO RATING VALUES (1003, 5);
INSERT INTO RATING VALUES (1004, 4);
INSERT INTO RATING VALUES (1005, 3);
```

STEP 4: QUERIES

1.List the titles of all movies directed

by'Hitchcock'.

```
SELECT MOV_TITLE
FROM MOVIES
WHERE DIR_ID IN ( SELECT DIR_ID
                  FROM DIRECTOR
                  WHERE DIR_NAME = 'HITCHCOCK');
```

O/P:

```
SQL> ( SELECT DIR_ID
2  FROM DIRECTOR
3  WHERE DIR_NAME = 'HITCHCOCK');
```

```
O/P:  DIR_ID
      -----
      61
```

```
SQL> SELECT MOV_TITLE
2  FROM MOVIES
3  WHERE DIR_ID IN ( SELECT DIR_ID
4  FROM DIRECTOR
5  WHERE DIR_NAME = 'HITCHCOCK');
```

```
O/P:
      MOV_TITLE
      -----
      AKASH
```

2. Find the movie names where one or more actors acted in two or more movies.

```
SQL> SELECT MOV_TITLE
2  FROM MOVIES M,
3  MOVIE_CAST MV
4  WHERE M.MOV_ID=MV.MOV_ID AND
5  ACT_ID IN (SELECT ACT_ID
```

```

6 FROM MOVIE_CAST
7 GROUP BY ACT_ID HAVING COUNT (ACT_ID)>1)
8 GROUP BY MOV_TITLE HAVING COUNT (*)>1;

```

O/P:

```

MOV_TITLE
-----
BAHUBALI-1

```

3. List all actors who acted in a movie before 2000 and also in a movie after 2015 (use JOINoperation).

```

SQL> SELECT ACT_NAME, MOV_TITLE, MOV_YEAR
2 FROM ACTOR A
3 JOIN MOVIE_CAST C ON A.ACT_ID=C.ACT_ID
4 JOIN MOVIES M ON C.MOV_ID=M.MOV_ID
5 WHERE M.MOV_YEAR NOT BETWEEN 2000 AND 2015;

```

O/P:

```

ACT_NAME      MOV_TITLE      MOV_YEAR
-----
ANUSHKA       BAHUBALI-2      2017

```

```

SQL> SELECT A.ACT_NAME, A.ACT_NAME, C.MOV_TITLE, C.MOV_YEAR
2 FROM ACTOR A, MOVIE_CAST B, MOVIES C
3 WHERE A.ACT_ID=B.ACT_ID AND B.MOV_ID=C.MOV_ID
4          AND C.MOV_YEAR NOT BETWEEN 2000 AND 2015;

```

O/P:

```

ACT_NAME      ACT_NAME      MOV_TITLE      MOV_YEAR
-----
ANUSHKA       ANUSHKA       BAHUBALI-2      2017

```

4. Find the title of movies and number of stars for each movie that has at least one rating and find the highest number of stars that movie received. Sort the result by movietitle.

```
SELECT MOV_TITLE, MAX (REV_STARS)
FROM MOVIES
INNER JOIN RATING USING (MOV_ID)
GROUP BY MOV_TITLE
HAVING MAX (REV_STARS)>0
ORDER BY MOV_TITLE;
```

O/P:

MOV_TITLE	MAX(REV_STARS)
AKASH	5
BAHUBALI-1	2
BAHUBALI-2	4
FF6	3
WAR HORSE	4

5. Update rating of all movies directed by 'Steven Spielberg' to 5

```
UPDATE RATING SET REV_STARS=5
WHERE MOV_ID IN ( SELECT MOV_ID
                  FROM MOVIES
                  WHERE DIR_ID IN ( SELECT DIR_ID
                                    FROM DIRECTOR
                                    WHERE DIR_NAME = 'STEVEN SPIELBERG'));
```

O/P:

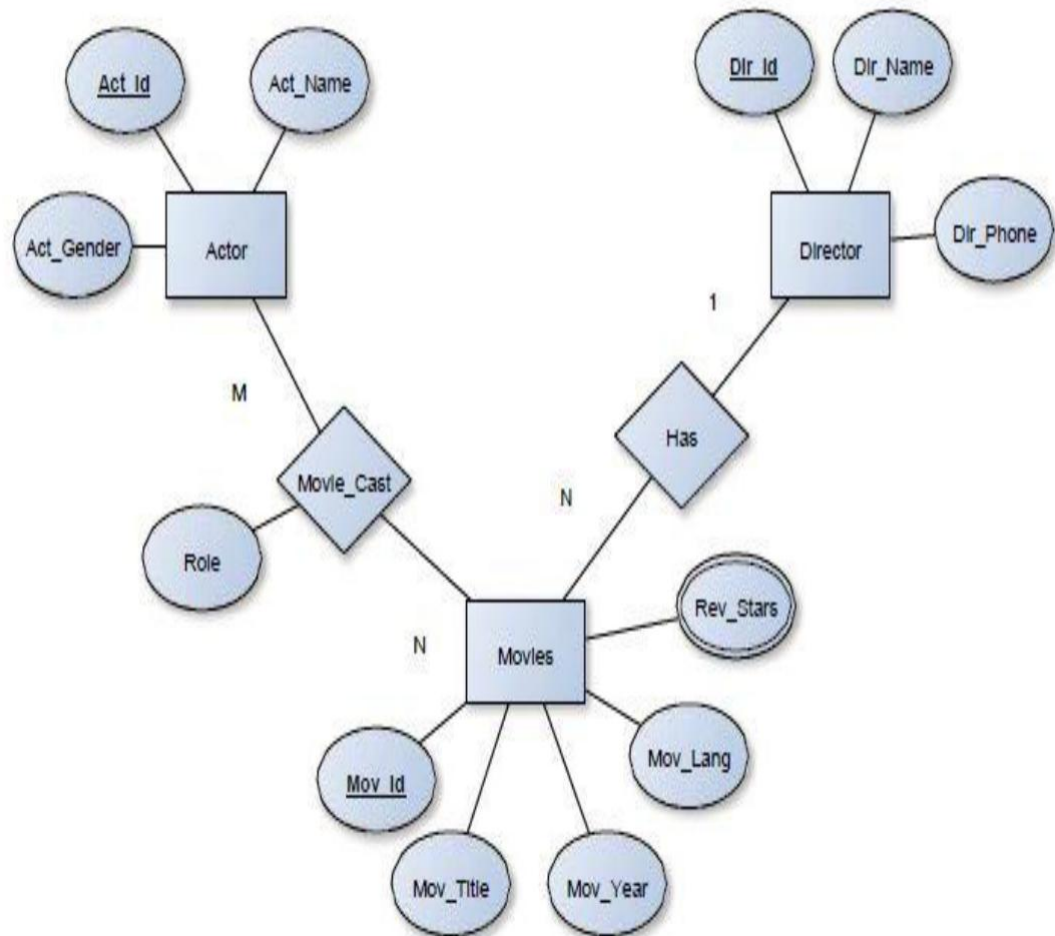
2 rows updated.

```
SQL> SELECT*FROM RATING;
```

```
MOV_ID REV_STARS
```

1001	4
1002	2
1003	5
1004	5
1005	5

Entity-Relationship Diagram



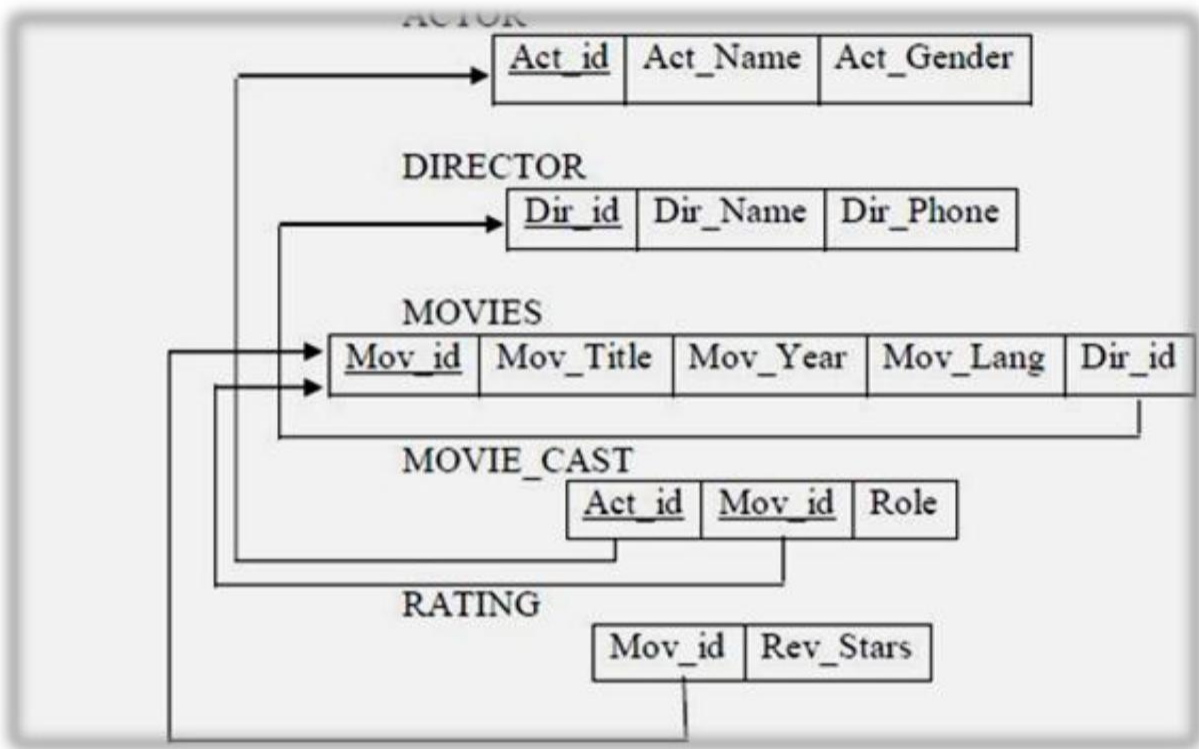


TABLE 6:

CREATE TABLE

CREATE TABLE SAILORS

```
CREATE TABLE SAILORS(  
    SID INT,  
    SNAME VARCHAR(20),  
    RATING INT,  
    AGE REAL,  
    PRIMARY KEY(SID));
```

CREATE TABLE BOATS

```
CREATE TABLE BOATS(  
    BID INT,  
    BNAME VARCHAR(20),  
    COLOR VARCHAR(20),  
    PRIMARY KEY (BID));
```

CREATE TABLE RESERVES

```
CREATE TABLE RESERVES(  
    SID INT,  
    BID INT,  
    DAY DATE,  
    PRIMARY KEY (BID,SID));
```

INSERT INTO SAILORS VALUES:

```
INSERT INTO SAILORS VALUES(22,'DUSTIN',7,45);  
INSERT INTO SAILORS VALUES(29,'BRUTUS',1,33);  
INSERT INTO SAILORS VALUES(31,'LUBBER',8,55.5);  
INSERT INTO SAILORS VALUES(32,'ANDY',8,25.5);  
INSERT INTO SAILORS VALUES(58,'RUSTY',10,35);  
INSERT INTO SAILORS VALUES(64,'HORATIO',7,35);  
INSERT INTO SAILORS VALUES(71,'ZORBA',10,16);  
INSERT INTO SAILORS VALUES(74,'HORATIO',9,40);  
INSERT INTO SAILORS VALUES(85,'ART',3,25.5);  
INSERT INTO SAILORS VALUES(95,'BOB',3,63.5);
```

INSERT INTO BOATS VALUES:

```
INSERT INTO BOATS VALUES(101,'INTERLAKE','BLUE');  
INSERT INTO BOATS VALUES(102,'INTERLAKE','RED');  
INSERT INTO BOATS VALUES(103,'CLIPPER','GREEN');  
INSERT INTO BOATS VALUES(104,'MARINE','RED');
```

INSERT INTO RESERVES VALUES:

```
INSERT INTO RESERVES VALUES(22,101,'10-OCT-1998');  
INSERT INTO RESERVES VALUES(22,102,'10-OCT-1998');  
INSERT INTO RESERVES VALUES(22,103,'08-OCT-1998');  
INSERT INTO RESERVES VALUES(22,104,'07-OCT-1998');
```

```

INSERT INTO RESERVES VALUES(31,102,'10-NOV-1998');

INSERT INTO RESERVES VALUES(31,103,'06-NOV-1998');

INSERT INTO RESERVES VALUES(31,104,'12-NOV-1998');

INSERT INTO RESERVES VALUES(64,101,'05-SEP-1998');

INSERT INTO RESERVES VALUES(64,102,'08-SEP-1998');

INSERT INTO RESERVES VALUES(74,103,'08-SEP-1998');

```

O/P:

```
SQL> SELECT*FROM SAILORS;
```

SID	SNAME	RATING	AGE
22	DUSTIN	7	45
29	BRUTUS	1	33
31	LUBBER	8	55.5
32	ANDY	8	25.5
58	RUSTY	10	35
64	HORATIO	7	35
71	ZORBA	10	16
74	HORATIO	9	40
85	ART	3	25.5
95	BOB	3	63.5

O/P:

```
SQL> SELECT*FROM BOATS;
```

BID	BNAME	COLOR
101	INTERLAKE	BLUE

102 INTERLAKE RED

103 CLIPPER GREEN

104 MARINE RED

O/P:

SQL> SELECT*FROM RESERVES;

SID	BID DAY
-----	---------

22	101 10-OCT-98
----	---------------

22	102 10-OCT-98
----	---------------

22	103 08-OCT-98
----	---------------

22	104 07-OCT-98
----	---------------

31	102 10-NOV-98
----	---------------

31	103 06-NOV-98
----	---------------

31	104 12-NOV-98
----	---------------

64	101 05-SEP-98
----	---------------

64	102 08-SEP-98
----	---------------

74	103 08-SEP-98
----	---------------

4. Simple SQL Query;

Ex1: Using DISTINCT:

SQL> SELECT DISTINCT S.sname, S.age

2 FROM sailors S;

SNAME	AGE
ANDY	25.5
RUSTY	35
BRUTUS	33
HORATIO	40
ART	25.5
DUSTIN	45
BOB	63.5
LUBBER	55.5
ZORBA	16
HORATIO	35

10 rows selected.

Ex2. Find all information of sailors who have reserved boat number 101.:

SQL> SELECT S.*

2 FROM Sailors S, Reserves R

3 WHERE S.sid = R.sid AND R.bid = 103;

SID	SNAME	RATING	AGE
22	DUSTIN	7	45
31	LUBBER	8	55.5

74 HORATIO

9 40

Ex3. Find the names of sailors who have reserved a red boat, and list in the order of age.:

```
SQL> SELECT S.sname, S.age
```

```
2 FROM Sailors S, Reserves R, Boats B
```

```
3 WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'RED'
```

```
4 ORDER BY S.age;
```

SNAME	AGE

HORATIO	35
DUSTIN	45
DUSTIN	45
LUBBER	55.5
LUBBER	55.5

Ex4. Find the names of sailors who have reserved at least one boat.:

```
SQL> SELECT sname
```

```
2 FROM Sailors S, Reserves R
```

```
3 WHERE S.sid = R.sid;
```

SNAME

DUSTIN
HORATIO

DUSTIN
LUBBER
HORATIO
DUSTIN
LUBBER
HORATIO
DUSTIN
LUBBER

10 rows selected.

Ex5. Find the ids and names of sailors who have reserved two different boats on the same day.:

```
SQL> SELECT DISTINCT S.sid, S.sname  
2 FROM Sailors S, Reserves R1, Reserves R2  
3 WHERE S.sid = R1.sid AND S.sid = R2.sid  
4 AND R1.day = R2.day AND R1.bid <> R2.bid;
```

SID SNAME

22 DUSTIN

Ex6. Using Expressions and Strings in the SELECT Command. :

```
SQL> SELECT SNAME,AGE,RATING+ 1 AS STH  
2 FROM SAILORS  
3 WHERE 2* RATING-1<10 AND SNAME LIKE'B_ %B';
```


no rows selected

5. Union, Intersect and Except:

IN and NOT

IN EXISTS and NOT EXISTS

UNIQUE and NOT UNIQUE

op ANY op ALL

```
SQL> SELECT R.sid
2 FROM Boats B, Reserves R
3 WHERE R.bid = B.bid AND B.color = 'RED'
4 UNION
5 SELECT R2.sid
6 FROM Boats B2, Reserves R2
7 WHERE R2.bid = B2.bid AND B2.color = 'GREEN';
```

SID

22

31

64

74

6. Nested Query:

EX8: Find the names of sailors who have reserved boat 103.:

```
SQL> SELECT S.sname
2 FROM Sailors S
3 WHERE S.sid IN ( SELECT R.sid
4 FROM Reserves R
5 WHERE R.bid = 103 ) ;
```

O/P:

SNAME

DUSTIN

LUBBER

HORATIO

(Correlated Nested Queries):

```
SQL> SELECT S.sname
2 FROM Sailors S
3 WHERE EXISTS ( SELECT *
4 FROM Reserves R
5 WHERE R.bid = 103
6 AND R.sid = S.sid ) ;
```

SNAME

DUSTIN

LUBBER

HORATIO

EX9: Find the name and the age of the youngest sailor.:

```
SQL> SELECT S.sname, S.age
2 FROM Sailors S
3 WHERE S.age <= ALL ( SELECT age
4 FROM Sailors ) ;
```

SNAME	AGE
-------	-----

ZORBA	16
-------	----

EX10: Find the names and ratings of sailor whose rating is better than some sailor called Horatio.:

```
SQL> SELECT S.sname, S.rating
2 FROM Sailors S
3 WHERE S.rating > ANY ( SELECT S2.rating
4 FROM Sailors S2
5 WHERE S2.sname = 'HORATIO' ) ;
```

SNAME	RATING
-------	--------

RUSTY	10
-------	----

ZORBA	10
HORATIO	9
LUBBER	8
ANDY	8

EX11: Find the names of sailors who have reserved all boats:

```
SQL> SELECT S.sname
2 FROM Sailors S
3 WHERE NOT EXISTS ( SELECT B.bid
4 FROM Boats B
5 WHERE NOT EXISTS ( SELECT R.bid
6 FROM Reserves R
7 WHERE R.bid = B.bid
8 AND R.sid = S.sid ) );
```

O/P:

SNAME

DUSTIN

Aggregation Operators:

COUNT ([DISTINCT] A): The number of (unique) values in the A column.

SUM ([DISTINCT] A): The sum of all (unique) values in the A column.

AVG ([DISTINCT] A): The average of all (unique) values in the A column.

MAX (A): The maximum value in the A column.

MIN (A): The minimum value in the A column.

EX12: Count the number of different sailor names.:

```
SQL> SELECT COUNT( DISTINCT S.sname )
```

```
2 FROM Sailors S;
```

```
COUNT(DISTINCTS.SNAME)
```

```
-----
```

```
9
```

EX13: Calculate the average age of all sailors.:

```
SQL> SELECT AVG(s.age)
```

```
2 FROM Sailors S
```

```
3 ;
```

```
AVG(S.AGE)
```

```
-----
```

```
37.4
```

EX14: Find the name and the age of the youngest sailor.:

```
SQL> SELECT S.sname, S.age
```

```
FROM Sailors S
```

```
WHERE S.age = (SELECT MIN(S2.age)
FROM Sailors S2 );
```

SNAME	AGE

ZORBA	16

EX15: Find the average age of sailors for each rating level:

```
SQL> SELECT S.rating, AVG(S.age) AS avg_age
FROM Sailors S
GROUP BY S.rating;
```

RATING	AVG_AGE

1	33
8	40.5
7	40
3	44.5
10	25.5
9	40

6 rows selected.

EX16: Find the average age of sailors for each rating level that has at least two sailors.:

```
SQL> SELECT S.rating, AVG(S.age) AS avg_age
```

```
FROM Sailors S

GROUP BY S.rating

HAVING COUNT(*) > 1;
```

```
RATING  AVG_AGE
```

```
-----
```

```
8    40.5
7     40
3    44.5
10   25.5
```

EX16: An example shows difference between WHERE and HAVING:

```
SQL> SELECT S.rating, AVG(S.age) as avg_age

FROM Sailors S

WHERE S.age >=40

GROUP BY S.rating;
```

```
RATING  AVG_AGE
```

```
-----
```

```
8    55.5
7     45
3    63.5
9     40
```

```
SQL> SELECT S.rating, AVG(S.age) as avg_age
FROM Sailors S
GROUP BY S.rating
HAVING AVG(S.age) >= 40;
```

RATING	AVG_AGE
--------	---------

8	40.5
7	40
3	44.5
9	40

5. NULL value and OUTER JOIN:

An example of OUTER JOIN:

```
SQL> SELECT sailors.sid, sailors.sname, reserves.bid
FROM sailors LEFT OUTER JOIN reserves ON reserves.sid = sailors.sid
ORDER BY sailors.sid;
```

SID	SNAME	BID
-----	-------	-----

22	DUSTIN	104
22	DUSTIN	101
22	DUSTIN	102
22	DUSTIN	103

29BRUTUS

31LUBBER 102

31LUBBER 104

31LUBBER 103

32 ANDY

58 RUSTY

64 HORATIO 101

SID SNAME	BID
-----------	-----

64 HORATIO	102
------------	-----

71 ZORBA

74 HORATIO	103
------------	-----

85 ART

95 BOB

16 rows selected.

TABLE 7:

--Create Table DEPARTMENT with PRIMARY KEY as DNO:

```
SQL> CREATE TABLE DEPT  
2 (DNO VARCHAR(20) PRIMARY KEY,  
3 DNAME VARCHAR(20),  
4 MGR_SSN VARCHAR(20),  
5 MGR_START_DATE DATE);
```

Table created.

--Create Table EMPLOYEE with PRIMARY KEY as SSN:

```
SQL> CREATE TABLE EMP  
2 (SSN VARCHAR(20) PRIMARY KEY,  
3 NAME VARCHAR(20),  
4 ADDRESS VARCHAR(20),  
5 SEX CHAR(1),  
6 SALARY INTEGER,  
7 SUPERSSN VARCHAR(20),  
8 DNO VARCHAR(20),  
9 FOREIGN KEY (SUPERSSN) REFERENCES emp (SSN),  
10 FOREIGN KEY (DNO) REFERENCES DEPT (DNO));
```

Table created.

-- ADD FOREIGN KEY Constraint to DEPARTMENT table:

```
SQL> ALTER TABLE DEPT
```

```
2 ADD FOREIGN KEY (MGR_SSN) REFERENCES EMPLOY(SSN);
```

Table altered.

--Create Table DLOCATION with PRIMARY KEY as DNO and DLOC and FOREIGN KEY DNO referring DEPARTMENT table:

```
SQL> CREATE TABLE DLOCATION
```

```
2 (DLOC VARCHAR(20),
```

```
3 DNO VARCHAR(20),
```

```
4 FOREIGN KEY (DNO) REFERENCES DEPT(DNO),
```

```
5 PRIMARY KEY (DNO, DLOC));
```

Table created.

```
SQL> DESC DLOCATION;
```

Name	Null?	Type
------	-------	------

DLOC	NOT NULL	VARCHAR2(20)
------	----------	--------------

DNO	NOT NULL	VARCHAR2(20)
-----	----------	--------------

--Create Table PROJECT with PRIMARY KEY as PNO and FOREIGN KEY DNO referring DEPARTMENT table:

```
SQL> CREATE TABLE PROJECT
```

```
2 (PNO INTEGER PRIMARY KEY,
```

```

3 PNAME VARCHAR(20),
4 PLOCATION VARCHAR(20),
5 DNO VARCHAR(20),
6 FOREIGN KEY (DNO) REFERENCES DEPT(DNO));

```

Table created.

```
SQL> DESC PROJECT;
```

Name	Null?	Type

PNO	NOT NULL	NUMBER(38)
PNAME		VARCHAR2(20)
PLOCATION		VARCHAR2(20)
DNO		VARCHAR2(20)

--Create Table WORKS_ON with PRIMARY KEY as PNO and SSN and FOREIGN KEY SSN and PNO referring EMPLOYEE and PROJECT table:

```

SQL> CREATE TABLE WORKS_ON
2 (HOURS INTEGER,
3 SSN VARCHAR(20),
4 PNO INTEGER,
5 FOREIGN KEY (SSN) REFERENCES EMPLOYE(SSN),
6 FOREIGN KEY (PNO) REFERENCES PROJECT(PNO),
7 PRIMARY KEY (SSN, PNO));

```

Table created.

```
SQL> DESC WORKS_ON;
```

Name	Null?	Type

HOURS		NUMBER(38)
SSN	NOT NULL	VARCHAR2(20)
PNO	NOT NULL	NUMBER(38)

--Inserting records into EMPLOYEE table:

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES
```

```
2 ('ABC01','BEN SCOTT','BANGALORE','M', 450000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES
```

```
2 ('ABC02','HARRY SMITH','BANGALORE','M', 500000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES
```

```
2 ('ABC03','LEAN BAKER','BANGALORE','M', 700000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES  
2 ('ABC04','MARTIN SCOTT','MYSORE','M', 500000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES  
2 ('ABC05','RAVAN HEGDE','MANGALORE','M', 650000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES  
2 ('ABC06','GIRISH HOSUR','MYSORE','M', 450000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES  
2 ('ABC07','NEELA SHARMA','BANGALORE','F', 800000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES  
2 ('ABC08','ADYA KOLAR','MANGALORE','F', 350000);
```

1 row created.

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES
```

```
2 ('ABC10','VEENA KUMARI','MYSORE','M',  
600000); 1 row created.
```

```
SQL> INSERT INTO EMPLOYEES(SSN, NAME, ADDRESS, SEX, SALARY) VALUES
```

```
2 ('ABC11','DEEPAK RAJ','BANGALORE','M', 500000);
```

1 row created.

```
SQL> SELECT * FROM EMPLOYEES;
```

SSN	NAME	ADDRESS	S	SALARY
ABC01	BEN SCOTT	BANGALORE	M	450000
ABC02	HARRY SMITH	BANGALORE	M	500000
ABC03	LEAN BAKER	BANGALORE	M	700000

SSN	NAME	ADDRESS	S	SALARY
ABC04	MARTIN SCOTT	MYSORE	M	500000

ABC05	RAVAN HEGDE	MANGALORE	M	650000
-------	-------------	-----------	---	--------

ABC06	GIRISH HOSUR	MYSORE	M	450000
-------	--------------	--------	---	--------

SSN	NAME	ADDRESS	S	SALARY
-----	------	---------	---	--------

SUPERSSN	DNO
----------	-----

ABC07	NEELA SHARMA	BANGALORE	F	800000
-------	--------------	-----------	---	--------

ABC08	ADYA KOLAR	MANGALORE	F	350000
-------	------------	-----------	---	--------

ABC10	VEENA KUMARI	MYSORE	M	600000
-------	--------------	--------	---	--------

SSN	NAME	ADDRESS	S	SALARY
-----	------	---------	---	--------

SUPERSSN	DNO
----------	-----

ABC11	DEEPAK RAJ	BANGALORE	M	500000
-------	------------	-----------	---	--------

10 rows selected.

--Inserting records into DEPARTMENT table:

SQL> INSERT INTO DEPT VALUES ('2','IT','ABC11', '04-FEB-2017'); 1 row created.

SQL> INSERT INTO DEPT VALUES ('3','HR','ABC01', '05-APR-2016'); 1 row created.

SQL> INSERT INTO DEPT VALUES ('4','HELPDESK', 'ABC10', '06-MAY-2017');

1 row created.

SQL> INSERT INTO DEPT VALUES ('5','SALES','ABC06', '01-JUN-2017'); 1 row created.

SQL> SELECT * FROM DEPT;

DNO	DNAME	MGR_SSN	MGR_START
2	IT	ABC11	04-FEB-17
3	HR	ABC01	05-APR-16
4	HELPDESK	ABC10	06-MAY-17
5	SALES	ABC06	01-JUN-17

--Updating EMPLOYEE records:

UPDATE EMP SET

SUPERSSN=NULL, DNO='3'

WHERE SSN='ABC01';

UPDATE EMP SET

SUPERSSN='ABC01', DNO='5'

WHERE SSN='ABC02';

UPDATE EMP SET

SUPERSSN='ABC04', DNO='5'

WHERE SSN='ABC03';

```
UPDATE EMPLOY SET  
SUPERSSN='ABC06', DNO='5'  
WHERE SSN='ABC04';
```

```
UPDATE EMPLOY SET  
DNO='5', SUPERSSN='ABC06'  
WHERE SSN='ABC05';
```

```
UPDATE EMPLOY SET  
DNO='5', SUPERSSN='ABC07'  
WHERE SSN='ABC06';
```

```
UPDATE EMPLOY SET  
DNO='5', SUPERSSN=NULL  
WHERE SSN='ABC07';
```

```
UPDATE EMPLOY SET  
DNO='1', SUPERSSN='ABC09'  
WHERE SSN='ABC08';
```

```
UPDATE EMPLOY SET  
DNO='1', SUPERSSN=NULL  
WHERE SSN='ABC09';
```

UPDATE EMPLOY SET

DNO='4', SUPERSSN=NULL

WHERE SSN='ABC10';

UPDATE EMPLOY SET

DNO='2', SUPERSSN=NULL

WHERE SSN='ABC11';

SELECT * FROM EMPLOY;

O/P:

SSN	NAME	ADDRESS	S	SALARY
-----	------	---------	---	--------

SUPERSSN	DNO
----------	-----

ABC01	BEN SCOTT	BANGALORE	M	450000
-------	-----------	-----------	---	--------

3

ABC02	HARRY SMITH	BANGALORE	M	500000
-------	-------------	-----------	---	--------

ABC01	5
-------	---

ABC03	LEAN BAKER	BANGALORE	M	700000
-------	------------	-----------	---	--------

ABC04	5
-------	---

SSN	NAME	ADDRESS	S	SALARY
-----	------	---------	---	--------

SUPERSSN	DNO
----------	-----

ABC04	MARTIN SCOTT	MYSORE	M	500000
-------	--------------	--------	---	--------

ABC06	5
-------	---

ABC05	RAVAN HEGDE	MANGALORE	M	650000
-------	-------------	-----------	---	--------

ABC06	5
-------	---

ABC06	GIRISH HOSUR	MYSORE	M	450000
-------	--------------	--------	---	--------

ABC07	5
-------	---

SSN	NAME	ADDRESS	S	SALARY
-----	------	---------	---	--------

SUPERSSN	DNO
----------	-----

ABC07	NEELA SHARMA	BANGALORE	F	800000
-------	--------------	-----------	---	--------

5

ABC08	ADYA KOLAR	MANGALORE	F	350000
-------	------------	-----------	---	--------

ABC10	VEENA KUMARI	MYSORE	M	600000
-------	--------------	--------	---	--------

4

SSN	NAME	ADDRESS	S	SALARY
-----	------	---------	---	--------

SUPERSSN	DNO
----------	-----

ABC11	DEEPAK RAJ	BANGALORE	M	500000
-------	------------	-----------	---	--------

2

10 rows selected.

--Inserting records into DLOCATION table:

```
INSERT INTO DLOCATION VALUES ('BENGALURU', '1');
```

```
INSERT INTO DLOCATION VALUES ('BENGALURU', '2');
```

```
INSERT INTO DLOCATION VALUES ('BENGALURU', '3');
```

```
INSERT INTO DLOCATION VALUES ('MYSORE', '4');
```

```
INSERT INTO DLOCATION VALUES ('MYSORE', '5');
```

```
SELECT * FROM DLOCATION;
```

O/P:

```
SQL> SELECT * FROM DLOCATION;
```

DLOC	DNO
------	-----

BENGALURU	2
-----------	---

BENGALURU	3
-----------	---

MYSORE	4
--------	---

MYSORE	5
--------	---

--Inserting records into PROJECT table:

```
INSERT INTO PROJECT VALUES (1000,'IOT','BENGALURU','5');
```

```
INSERT INTO PROJECT VALUES (1001,'CLOUD','BENGALURU','5');
```

```
INSERT INTO PROJECT VALUES (1002,'BIGDATA','BENGALURU','5');
```

```
INSERT INTO PROJECT VALUES (1003,'SENSORS','BENGALURU','3');
```

```

INSERT INTO PROJECT VALUES (1004,'BANK MANAGEMENT','BENGALURU','1');
INSERT INTO PROJECT VALUES (1005,'SALARY MANAGEMENT','BANGALORE','1');
INSERT INTO PROJECT VALUES (1006,'OPENSTACK','BENGALURU','4'); INSERT
INTO PROJECT VALUES (1007,'SMART CITY','BENGALURU','2');

```

```
SQL> SELECT * FROM PROJECT;
```

O/P:

PNO	PNAME	PLOCATION	DNO
1000	IOT	BENGALURU	5
1001	CLOUD	BENGALURU	5
1002	BIGDATA	BENGALURU	5
1003	SENSORS	BENGALURU	3
1006	OPENSTACK	BENGALURU	4
1007	SMART CITY	BENGALURU	2
1004	BANK MANAGEMENT	BENGALURU	1
1005	SALARY MANAGEMENT	BANGALORE	1

8 rows selected.

--Inserting records into WORKS_ON table

```
INSERT INTO WORKS_ON VALUES (4, 'ABC02', 1000);
```

```
INSERT INTO WORKS_ON VALUES (6, 'ABC02', 1001);
```

```

INSERT INTO WORKS_ON VALUES (8, 'ABC02', 1002);
INSERT INTO WORKS_ON VALUES (10, 'ABC03', 1000);
INSERT INTO WORKS_ON VALUES (3, 'ABC05', 1000);
INSERT INTO WORKS_ON VALUES (4, 'ABC06', 1001);
INSERT INTO WORKS_ON VALUES (5, 'ABC07', 1002);
INSERT INTO WORKS_ON VALUES (6, 'ABC04', 1002);
INSERT INTO WORKS_ON VALUES (7, 'ABC01', 1003);
INSERT INTO WORKS_ON VALUES (5, 'ABC08', 1004);
INSERT INTO WORKS_ON VALUES (6, 'ABC09', 1005);
INSERT INTO WORKS_ON VALUES (4, 'ABC10', 1006);
INSERT INTO WORKS_ON VALUES (10, 'ABC11', 1007);

```

```
SQL> SELECT * FROM WORKS_ON;
```

HOURS	SSN	PNO
4	ABC02	1000
6	ABC02	1001
8	ABC02	1002
10	ABC03	1000
3	ABC05	1000
4	ABC06	1001
5	ABC07	1002
6	ABC04	1002

7 ABC01 1003

5 ABC08 1004

4 ABC10 1006

HOURS SSN PNO

10 ABC11 1007

12 rows selected.

--Make a list of all project numbers for projects that involve an employee whose last name is 'Scott', either as a worker or as a manager of the department that controls the project.:

```
SQL> (SELECT DISTINCT P.PNO
2  FROM PROJECT P, DEPT D, EMPLOY E
3  WHERE P.DNO=D.DNO
4  AND D.MGR_SSN=E.SSN
5  AND E.NAME LIKE '%SCOTT')
6 UNION
7 (SELECT DISTINCT P1.PNO
8  FROM PROJECT P1, WORKS_ON W, EMPLOY E1
9  WHERE P1.PNO=W.PNO
10 AND E1.SSN=W.SSN
11 AND E1.NAME LIKE '%SCOTT');
```

O/P:

PNO

1002

1003

--Show the resulting salaries if every employee working on the 'IoT' project is given a 10 percent raise.:

```
SQL> SELECT E.NAME, 1.1*E.SALARY AS INCR_SAL
2 FROM EMP E, WORKS_ON W, PROJECT P
3 WHERE E.SSN=W.SSN
4 AND W.PNO=P.PNO
5 AND P.PNAME='IOT';
```

NAME	INCR_SAL

HARRY SMITH	550000
LEAN BAKER	770000
RAVAN HEGDE	715000

--Find the sum of the salaries of all employees of the 'Accounts' department, as well as the maximum salary, the minimum salary, and the average salary in this department

```
SQL> SELECT SUM(E.SALARY), MAX(E.SALARY), MIN(E.SALARY), AVG(E.SALARY)
2 FROM EMP E, DEPT D
3 WHERE E.DNO=D.DNO
4 AND D.DNAME='ACCOUNTS';
```

SUM(E.SALARY) MAX(E.SALARY) MIN(E.SALARY) AVG(E.SALARY)

350000 350000 350000 350000

--Retrieve the name of each employee who works on all the projects controlled by department number 5 (use NOT EXISTS operator).:

SQL> SELECT NAME

2 FROM EMPLOYEES

3 WHERE NOT EXISTS(SELECT PNO FROM PROJECT WHERE DNO= 5 AND PNO NOT IN (SELECT
PNO FROM WORKS_ON

4 WHERE E.SSN=SSN));

NAME

HARRY SMITH

--For each department that has more than five employees, retrieve the department number and the number of its employees who are making more than Rs. 6,00,000.:

SQL> SELECT D.DNO, COUNT (*)

FROM DEPT D, EMPLOYEES E

WHERE D.DNO=E.DNO

AND E.SALARY > 600000

AND D.DNO IN (SELECT E1.DNO

FROM EMPLOYEES E1

GROUP BY E1.DNO

HAVING COUNT(*)>5)

GROUP BY D.DNO;

DNO	COUNT (*)
-----	-----------

5	3
---	---